

ISE 105 - Programlamaya Giriş

İşaretçiler (Pointers)

Dr. Öğr. Üyesi Muhammed Kotan

Bilişim Sistemleri Mühendisliği

2025 Güz



İçindekiler

- 1 İşaretçilere Giriş
- 2 İşaretçi Tanımlama
- 3 İşaretçiler ve Fonksiyonlar
- 4 İşaretçiler ve Diziler
- 5 Dinamik Bellek Yönetimi
- 6 Null Pointer ve Pointer Hataları
- 7 Const ve İşaretçiler
- 8 Çift İşaretçiler
- 9 Uygulamalar
- 10 Alıştırmalar
- 11 Özet

İşaretçi (Pointer) Nedir?

Tanım

İşaretçi, başka bir değişkenin **bellek adresini** tutan özel bir değişkendir.

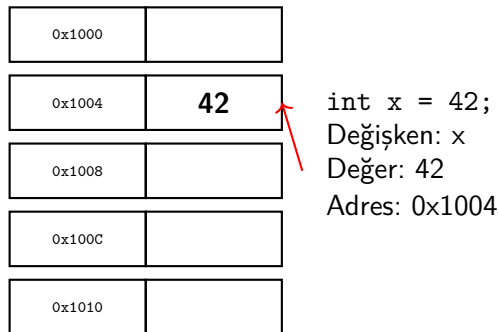
Neden Kullanılır?

- Dinamik bellek yönetimi
- Büyük veri yapılarını verimli paylaşma
- Fonksiyonlarda değer değiştirme
- Diziler ve stringlerle çalışma
- Veri yapıları (linked list, tree...)

Bellek Kavramı:

- Her değişken bellekte bir yerde saklanır
- Her bellek hücresinin bir adresi vardır
- İşaretçiler bu adresleri saklar
- Adres üzerinden değere erişilebilir

Bellek Adresi Kavramı



Analoji: Bellek adresi = Ev adresi, Değişken = Evin içindeki eşyalar

İşaretçi Tanımlama ve Operatörler

Operatörler:

- `*` → İşaretçi bildirimi ve değere erişim
- `&` → Adres operatörü

Sözdizimi:

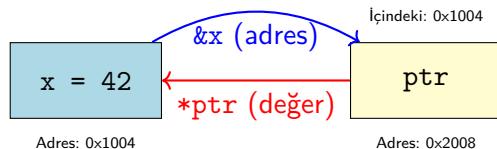
```
tip *isaretci_adi;
```

```
int *ptr;           // int isaretci
double *dptr;       // double isaretci
char *cptr;         // char isaretci
```

Temel Örnek:

```
int x = 42;
int *ptr;           // Isaretci tanimi
ptr = &x;           // x'in adresini ata

cout << "x'in degeri: "
      << x << endl;
cout << "x'in adresi: "
      << &x << endl;
cout << "ptr'deki adres: "
      << ptr << endl;
cout << "ptr'nin gosterdigi: "
      << *ptr << endl;
```



& (Address-of)

- Değişkenin adresini verir
- `&x` → `x`'in adresi

* (Dereference)

- Adresteki değere erişir
- `*ptr` → `ptr`'nin gösterdiği değer

Örnek 1: Temel Kullanım

```
#include <iostream>
using namespace std;

int main() {
    int sayi = 100;
    int *ptr = &sayi;

    cout << "sayi: " << sayi << endl;
    cout << "&sayi: " << &sayi << endl;
    cout << "ptr: " << ptr << endl;
    cout << "*ptr: " << *ptr << endl;

    // İşaretçi ile değiştirme
    *ptr = 200;
    cout << "sayi: " << sayi << endl;

    return 0;
}
```

Çıktı (Örnek):

```
sayi: 100
&sayi: 0x7ffc1234
ptr: 0x7ffc1234
*ptr: 100
sayi: 200
```

Açıklama:

- ptr ve &sayi aynı
- *ptr = 200 ile sayi değişti
- İşaretçi üzerinden orijinal değişken değişir

Fonksiyonlarda İşaretçi Kullanımı

Neden Kullanılır?

Fonksiyonların orijinal değişkenleri değiştirmesini sağlar (pass by pointer).

Değer ile (Çalışmaz):

```
void degistir(int x) {  
    x = 100;  
}  
  
int main() {  
    int sayi = 50;  
    degistir(sayi);  
    cout << sayi; // 50  
    return 0;  
}
```

İşaretçi ile (Çalışır):

```
void degistir(int *ptr) {  
    *ptr = 100;  
}  
  
int main() {  
    int sayi = 50;  
    degistir(&sayi);  
    cout << sayi; // 100  
    return 0;  
}
```

Not: İşaretçi ile referans benzer çalışır, ancak işaretçi esnektir (NULL olabilir, değiştirilebilir).

İşaretçi Parametrelili Fonksiyon Örnekleri

```
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void arttir(int *sayi) {
    (*sayi)++;
}

int main() {
    int x = 10, y = 20;
    cout << "Once: x=" << x << ", y=" << y << endl;

    swap(&x, &y);
    cout << "Swap sonrasi: x=" << x << ", y=" << y << endl;

    arttir(&x);
    cout << "Arttir sonrasi: x=" << x << endl;

    return 0;
}
```

Önemli Bilgi

Dizi adı aslında ilk elemanın adresini tutan bir işaretçidir!

```
#include <iostream>
using namespace std;

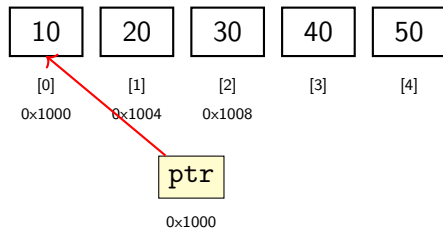
int main() {
    int dizi[5] = {10, 20, 30, 40, 50};
    int *ptr = dizi; // dizi = &dizi[0]

    cout << "dizi: " << dizi << endl;
    cout << "&dizi[0]: " << &dizi[0] << endl;
    cout << "ptr: " << ptr << endl;

    // Elemanlara erişim
    cout << "dizi[0]: " << dizi[0] << endl;
    cout << "*ptr: " << *ptr << endl;
    cout << "*(ptr+1): " << *(ptr+1) << endl; // 20
    cout << "ptr[2]: " << ptr[2] << endl;    // 30

    return 0;
}
```

Dizi ve İşaretçi İlişkisi



```
int dizi[5] = {10,20,30,40,50};  
int *ptr = dizi;  
*ptr → 10  
*(ptr+1) → 20  
ptr[2] → 30
```

İşaretçi Aritmetiği:

```
int dizi[5] = {10, 20, 30, 40, 50};
int *ptr = dizi;

// Erisim yontemleri
cout << dizi[0];    // 10
cout << *ptr;       // 10
cout << *(ptr+0);   // 10

cout << dizi[2];    // 30
cout << *(ptr+2);   // 30
cout << ptr[2];     // 30

// Isaretci ilerleme
ptr++;              // Sonraki eleman
cout << *ptr;       // 20

ptr += 2;           // 2 eleman ileri
cout << *ptr;       // 40
```

Dizi Dolaşma:

```
int dizi[5] = {10, 20, 30, 40, 50};
int *ptr = dizi;

// Yontem 1: Indeks
for (int i = 0; i < 5; i++) {
    cout << dizi[i] << " ";
}

// Yontem 2: Isaretci
for (int i = 0; i < 5; i++) {
    cout << *(ptr+i) << " ";
}

// Yontem 3: Isaretci ilerleme
for (int *p=dizi; p<dizi+5; p++) {
    cout << *p << " ";
}
```

new ve delete Operatörleri

- new → Dinamik bellek ayırır (heap)
- delete → Ayrılan belleği serbest bırakır

Tek Değişken:

```
// Bellek ayır
int *ptr = new int;
*ptr = 42;

cout << *ptr << endl;

// Bellek serbest bırak
delete ptr;
ptr = nullptr; // Güvenli
```

Dizi:

```
// Dizi için bellek ayır
int *dizi = new int[5];

for (int i = 0; i < 5; i++) {
    dizi[i] = i * 10;
}

// Bellek serbest bırak
delete[] dizi; // [] kullan!
dizi = nullptr;
```

Önemli: Her new için mutlaka delete, her new[] için delete[] kullanın!

Stack vs Heap Bellek

Stack

Yerel değişkenler
Otomatik yönetim
Hızlı
Sınırlı boyut
`int x = 10;`
`int arr[10];`

Heap

Dinamik değişkenler
Manuel yönetim
Daha yavaş
Büyük boyut
`new int;`
`new int[100];`

Ne Zaman Heap Kullanılır?

- Boyut çalışma zamanında belirleniyorsa
- Büyük veri yapıları
- Fonksiyon dışında kullanılacak veriler

Dinamik Dizi Örneği

```
#include <iostream>
using namespace std;

int main() {
    int boyut;
    cout << "Dizi boyutunu girin: ";
    cin >> boyut;

    // Dinamik dizi olustur
    int *dizi = new int[boyut];

    // Doldur
    for (int i = 0; i < boyut; i++) {
        dizi[i] = i * i;
    }

    // Yazdir
    for (int i = 0; i < boyut; i++) {
        cout << dizi[i] << " ";
    }
    cout << endl;

    // Bellek serbest birak
    delete[] dizi;

    return 0;
}
```

Null Pointer

nullptr (C++11)

Hiçbir yeri göstermeyen işaretçi değeri.

Kullanımı:

```
int *ptr = nullptr;

// Kontrol et
if (ptr == nullptr) {
    cout << "Gecersiz" << endl;
}

// Güvenli kullanım
if (ptr != nullptr) {
    cout << *ptr << endl;
}

// Delete sonrası
delete ptr;
ptr = nullptr; // Güvenli
```

Yaygın Hatalar:

```
// HATA 1: İlk deger vermeden
int *ptr;
*ptr = 10; // TEHLIKELI!

// HATA 2: Delete sonrası
int *ptr = new int(5);
delete ptr;
*ptr = 10; // HATA!

// HATA 3: Cift delete
delete ptr;
delete ptr; // HATA!

// DOGRU:
int *ptr = nullptr;
if (ptr != nullptr) {
    *ptr = 10;
}
```


Yaygın Hatalar

- ❶ **Dangling Pointer:** Delete edilen belleğe erişim
- ❷ **Memory Leak:** Delete edilmeyen bellek
- ❸ **Wild Pointer:** İlk değer verilmemiş işaretçi
- ❹ **Segmentation Fault:** Geçersiz belleğe erişim

İyi Pratikler:

- Her zaman işaretçilere ilk değer verin (`nullptr`)
- Her `new` için `delete` kullanın
- Delete sonrası `nullptr` atayın
- Kullanmadan önce `nullptr` kontrolü yapın
- Akıllı işaretçiler kullanın (modern C++)

Const İşaretçiler

Üç Farklı Kullanım

- 1 Const değere işaretçi
- 2 Const işaretçi
- 3 Her ikisi de const

// 1. Const degere isaretci (deger degismez)

```
const int *ptr1 = &x;
```

```
*ptr1 = 10;    // HATA!
```

```
ptr1 = &y;     // OK
```

// 2. Const isaretci (adres degismez)

```
int *const ptr2 = &x;
```

```
*ptr2 = 10;    // OK
```

```
ptr2 = &y;     // HATA!
```

// 3. Her ikisi de const

```
const int *const ptr3 = &x;
```

```
*ptr3 = 10;    // HATA!
```

```
ptr3 = &y;     // HATA!
```

Const İşaretçi Kullanım Alanları

Kullanım	Sözdizimi	Açıklama
Değer korumalı	<code>const int *ptr</code>	Fonksiyonlarda veri koruma
Adres sabit	<code>int *const ptr</code>	Değiştirilebilir veri, sabit bağlantı
Her ikisi	<code>const int *const ptr</code>	Tam koruma

Örnek Kullanım:

- `void print(const int *arr, int size)` → Diziyi değiştirmeden yazdır
- `const char *str = "Hello"` → String literalleri

Pointer to Pointer (Çift İşaretçi)

Tanım

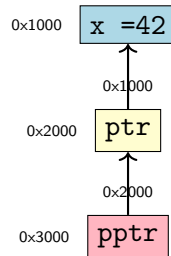
Başka bir işaretçinin adresini tutan işaretçi.

```
#include <iostream>
using namespace std;
int main() {
    int x = 42;
    int *ptr = &x;
    int **pptr = &ptr;

    cout << "x: " << x << endl;
    cout << "*ptr: " << *ptr << endl;
    cout << "**pptr: " << **pptr << endl;

    **pptr = 100;
    cout << "x: " << x << endl;
    return 0;
}
```

Bellek Düzeni:



Çift İşaretçi Örneği 1: 2D Dinamik Dizi

Kod:

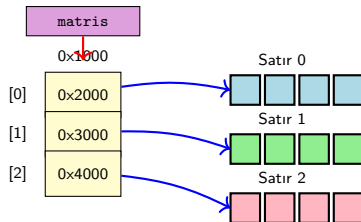
```
int satir = 3, sutun = 4;

// 2D dizi olustur
int **matris = new int*[satir];
for (int i = 0; i < satir; i++) {
    matris[i] = new int[sutun];
}

// Doldur
for (int i = 0; i < satir; i++) {
    for (int j = 0; j < sutun; j++) {
        matris[i][j] = i * j;
    }
}

// Serbest bırak
for (int i = 0; i < satir; i++) {
    delete[] matris[i];
}
delete[] matris;
```

Bellekte Yapı:



Açıklama:

- `matris` → İşaretçiler dizisi
- `matris[i]` → Her satırın adresi
- `matris[i][j]` → Hücre değeri

Çift İşaretçi Örneği 2: Fonksiyonda İşaretçi Değiştirme

Kod:

```
void bellekAyir(int **ptr, int boyut) {  
    *ptr = new int[boyut];  
}  
  
int main() {  
    int *dizi = nullptr;  
  
    bellekAyir(&dizi, 10);  
  
    // Kullan  
    for (int i = 0; i < 10; i++) {  
        dizi[i] = i;  
    }  
  
    delete[] dizi;  
    return 0;  
}
```

Anahtar Nokta:

- &dizi → Adres gönderilir
- **ptr → Çift işaretçi alır
- *ptr = new → Orijinali değiştirir

Adım Adım:

1. Başlangıç:

0x7000

dizi: nullptr

2. Çağrı:

0x7000

dizi: nullptr

ptr: 0x7000

0x8000

3. *ptr = new:

0x7000

dizi: 0x9000

ptr: 0x7000

0x8000

0x9000

new dizi[10]

Sonuç: Main'deki dizi artık 0x9000'i gösteriyor!

Uygulama 1: Swap Fonksiyonu

İşaretçi ile:

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;

    cout << "Önce: x=" << x
          << ", y=" << y << endl;

    swap(&x, &y);

    cout << "Sonra: x=" << x
          << ", y=" << y << endl;

    return 0;
}
```

Referans ile:

```
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;

    cout << "Önce: x=" << x
          << ", y=" << y << endl;

    swap(x, y);

    cout << "Sonra: x=" << x
          << ", y=" << y << endl;

    return 0;
}
```

Fark: İşaretçi kullanımında & operatörü gerekli, referans daha temiz görünür.

Uygulama 2: Dizi İşlemleri

```
#include <iostream>
using namespace std;

int diziToplam(int *dizi, int boyut) {
    int toplam = 0;
    for (int i = 0; i < boyut; i++) {
        toplam += dizi[i]; // veya *(dizi+i)
    }
    return toplam;
}

void diziTersCevir(int *dizi, int boyut) {
    for (int i = 0; i < boyut/2; i++) {
        int temp = dizi[i];
        dizi[i] = dizi[boyut-1-i];
        dizi[boyut-1-i] = temp;
    }
}

int main() {
    int sayilar[] = {10, 20, 30, 40, 50};
    int boyut = 5;

    cout << "Toplam: " << diziToplam(sayilar, boyut) << endl;

    diziTersCevir(sayilar, boyut);
    for (int i = 0; i < boyut; i++) {
        cout << sayilar[i] << " ";
    }

    return 0;
}
```


Uygulama 3: Dinamik String İşlemleri

Fonksiyonlar:

```
#include <iostream>
#include <cstring>
using namespace std;

char* stringKopyala(const char *kaynak) {
    int uzunluk = strlen(kaynak);
    char *hedef = new char[uzunluk + 1];
    // Manuel kopyalama
    for (int i = 0; i <= uzunluk; i++) {
        hedef[i] = kaynak[i];
    }
    return hedef;
}

char* stringBirlestir(const char *s1, const char *s2) {
    int u1 = strlen(s1);
    int u2 = strlen(s2);
    char *sonuc = new char[u1 + u2 + 1];
    // s1'i kopyala
    for (int i = 0; i < u1; i++) {
        sonuc[i] = s1[i];
    }
    // s2'yi ekle
    for (int i = 0; i <= u2; i++) {
        sonuc[u1 + i] = s2[i];
    }
    return sonuc;
}
```

Kullanım:

```
int main() {
    char *str1 = stringKopyala("Merhaba ");
    char *str2 = stringKopyala("Dunya!");
    char *birlesmis = stringBirlestir(str1, str2);

    cout << birlesmis << endl;
    // Cikti: Merhaba Dunya!

    // Bellek temizleme
    delete[] str1;
    delete[] str2;
    delete[] birlesmis;

    return 0;
}
```

Not:

- Manuel karakter kopyalama
- '\0' dahil kopyalanır
- Her new[] için delete[]

Uygulama 4: Fonksiyondan Dizi Döndürme

```
#include <iostream>
using namespace std;

int* fibonacci(int n) {
    int *dizi = new int[n];

    if (n >= 1) dizi[0] = 0;
    if (n >= 2) dizi[1] = 1;

    for (int i = 2; i < n; i++) {
        dizi[i] = dizi[i-1] + dizi[i-2];
    }

    return dizi;
}

int main() {
    int boyut = 10;
    int *fib = fibonacci(boyut);

    cout << "Fibonacci serisi: ";
    for (int i = 0; i < boyut; i++) {
        cout << fib[i] << " ";
    }
    cout << endl;

    delete[] fib;

    return 0;
}
```

Alıştırma Soruları - 1

- 1 İki sayının değerlerini değiştiren swap fonksiyonunu işaretçi kullanarak yazın
- 2 Bir dizideki en büyük elemanın adresini döndüren fonksiyon yazın
- 3 Dinamik olarak boyutu kullanıcıdan alınan bir dizi oluşturun ve doldurun
- 4 İki diziyi birleştiren (concatenate) fonksiyon yazın (dinamik bellek)
- 5 Bir string'in uzunluğunu işaretçi kullanarak hesaplayan fonksiyon
- 6 Dizi elemanlarını ters çeviren fonksiyon
- 7 İşaretçi kullanarak iki string'i karşılaştıran fonksiyon
- 8 Bir sayı dizisinin kopyasını oluşturan fonksiyon (new kullanarak)

Alıştırma Soruları - 2

- 2D dinamik matris oluşturan ve serbest bırakan fonksiyonlar yazın
- Bir dizideki çift sayıları yeni bir diziye kopyalayan fonksiyon
- İşaretçi kullanarak palindrom kontrolü yapan fonksiyon
- String içindeki boşlukları kaldıran fonksiyon (dinamik)
- İki matrisin çarpımını hesaplayan fonksiyon (dinamik bellek)
- Dinamik olarak linked list düğümü oluşturan yapı

İleri Seviye:

- Dinamik olarak büyüyen dizi yapısı (vector benzeri)
- İşaretçi kullanarak basit bellek havuzu (memory pool)
- Çift işaretçi ile 3D dizi oluşturma

Temel Kavramlar:

- İşaretçi = Bellek adresi
- & → Adres al
- * → Değere eriş
- nullptr → Boş işaretçi

Dinamik Bellek:

- new → Ayır
- delete → Serbest bırak
- new[] → Dizi ayır
- delete[] → Dizi serbest bırak

Kullanım Alanları:

- Fonksiyonlarda değer değiştirme
- Dizilerle çalışma
- Dinamik veri yapıları
- Bellek yönetimi

Dikkat Edilecekler:

- Memory leak'ten kaçının
- Dangling pointer'dan sakının
- Her new için delete
- nullptr kontrolü yapın

Hatırlatmalar

- 1 **İlk Değer Verin:** `int *ptr = nullptr;`
- 2 **Kontrol Edin:** `if (ptr != nullptr)`
- 3 **Temizleyin:** Her new için delete
- 4 **Güvenli Yapın:** Delete sonrası `ptr = nullptr;`
- 5 **Dizi Dikkat:** `new[]` için `delete[]`
- 6 **Modern C++:** Akıllı işaretçiler kullanın (`unique_ptr`, `shared_ptr`)

Yaygın Hatalar:

- İlk değer vermeden kullanma
- Delete ettikten sonra kullanma
- Memory leak (delete unutma)
- delete yerine `delete[]` kullanmama

Soru&Cevap

Teşekkürler!